SCOPE OF WORK

Hermes Rebuild

A modular, queue-driven rebuild of the Hermes SMS platform — target end-state architecture, requirements, and deliverables.

PREPARED BY

Carson Craig

DATE

May 19, 2026

PROJECT

Hermes (txt.cx) infrastructure rebuild

DOCUMENT TYPE

End-state architecture spec

CURRENT STACK

Next.js 14 · Prisma · Postgres · Telnyx

TARGET THROUGHPUT

60 MPS (20 × 3 toll-free numbers)

ONE-PARAGRAPH SUMMARY

Hermes is my personal SMS marketing app. It works, but it can't hit the 20 MPS toll-free throughput limit on three numbers at once, and the dashboard is slow to load because every page does too much aggregate work against Postgres. The fix is structural: move the send loop off the web process onto purpose-built worker droplets, put Redis between submission and Telnyx as a real per-number rate limiter, and split Postgres into an OLTP primary plus a read replica that absorbs all dashboard/analytics traffic. This document describes the end-state system to build, what it must do, how I'll know it's done, and what's intentionally out of scope.

1 Project Context

What Hermes does today, why a rebuild is needed, and the shape of the end state.

What Hermes is

Hermes is a Next.js 14 (App Router) application I use to send promotional SMS messages to my opt-in subscriber list via the Telnyx API. It manages contacts, campaigns, click-tracked shortlinks, conversations, revenue attribution, and a "revolver" pool of toll-free phone numbers. It's deployed on DigitalOcean App Platform with a single managed Postgres instance behind it.

The two problems we're solving

1. **Throughput.** Telnyx allows 20 messages per second per toll-free number. With three numbers configured, the theoretical aggregate is 60 MPS. In practice the system tops out around 5–10 MPS sustained before Postgres connection pool exhaustion and webhook backpressure compound. The send loop, every webhook, and the dashboard all run inside the same Next.js process against the same 20-connection pool.
2. **Dashboard load time.** Every page fires 5–11 unbounded aggregate queries against Postgres on each load. Conversations are re-derived in JavaScript on every paint. There is no caching layer, no read replica, and no rollup tables. As the message volume grows, the dashboard gets slower.

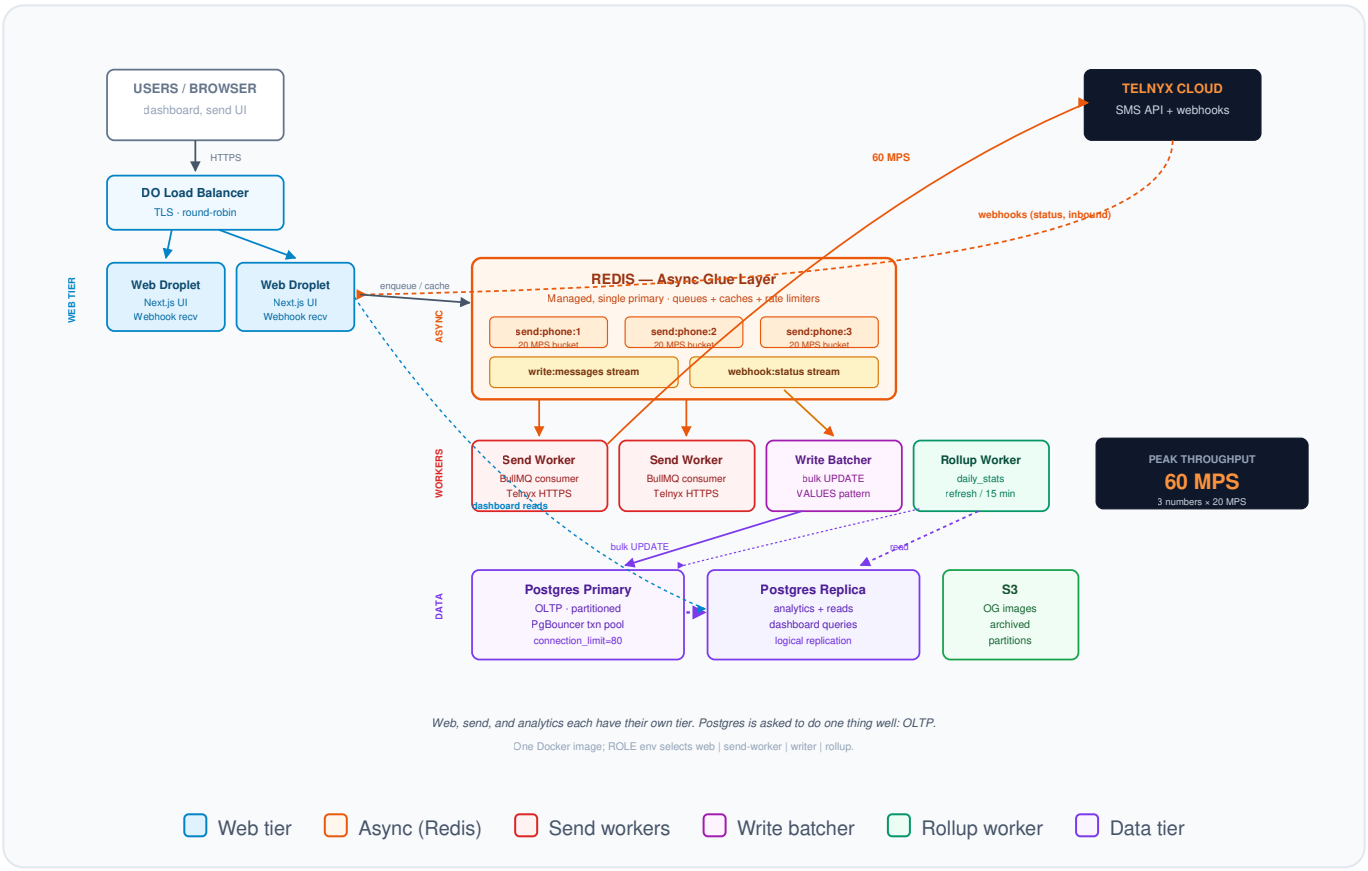
The end-state vision

One Docker image, deployed to standardized DigitalOcean droplets behind a load balancer. The image runs in one of several modes via a `ROLE` environment variable: **web**, **send-worker**, **writer**, **rollup**. Redis sits between web and workers as the queue, rate limiter, and hot-read cache. Postgres splits into an OLTP primary and a read replica. Add capacity by adding droplets, not by rewriting.

> **The data model is mostly right; the runtime is wrong.** This SOW is not a from-scratch rewrite of the database schema or the feature set. It's a re-platforming of the send pipeline and the read path. Existing features — revolver assignment, click codes, shortlinks, opt-out handling, scheduled sends, workflow campaigns, delivery webhooks, spam-rate analytics, conversation view — must all continue to work.

2 End-State Architecture

The system to build, drawn as one picture.



How the data flows

1. **Send submission** — UI hits a server action on a web droplet. The action streams contact IDs into a Redis set, creates a `SendJob` row in Postgres for UI tracking, and enqueues a single BullMQ job. The HTTP request returns immediately.
2. **Fan-out** — A campaign-fanout worker reads the contact IDs in chunks, groups them by assigned phone number, generates click codes in bulk, and pushes per-message jobs onto per-number Redis queues (`send:phone:1`, `send:phone:2`, `send:phone:3`) each gated by a 20-MPS Redis token bucket.
3. **Send execution** — Send-worker droplets consume from the per-number queues. Each job is one Telnyx HTTPS call with an idempotency key. On response, the worker pushes the result onto a shared `write:messages` Redis stream. The worker never writes to Postgres directly.
4. **Write batching** — A write-batcher droplet drains the `write:messages` and `webhook:status` streams in batches of ~200 every 200 ms and applies one `UPDATE … FROM (VALUES …)` per batch to Postgres.
5. **Webhooks** — Telnyx posts status updates to web droplets. Receiver writes the payload to `webhook:status` and returns 200 in <10 ms. The write-batcher applies them.
6. **Reads** — Dashboard, analytics, and list pages read from a Postgres read replica, with Redis caching summary blobs (dashboard, message-stats, campaign list) and a `Conversation` table that's maintained on every message write so the messages view never re-groups in JS.

3 Functional Requirements

What the new system must do. Every existing feature in Hermes today must continue to work; this section enumerates the ones with non-trivial implementation considerations under the new architecture.

FR-01

Send via per-number rate-limited queues

Every outbound message goes through a Redis queue scoped to its assigned phone number. The queue enforces a token-bucket rate limit (default 20 MPS, configurable per number via `PhoneNumber.rateLimit`). Bursts above the limit are queued, not dropped.

FR-02

Revolver assignment on opt-in

New contacts are assigned to a phone number from the user's `PhoneNumber` pool using a least-loaded heuristic among numbers with `acceptingNewLeads = true`. Existing contacts retain their assignment.

FR-03

Telnyx idempotency

Every Telnyx `messages.create` request includes an `Idempotency-Key` header equal to the pre-generated message UUID. Retries (network, 5xx, timeout) never produce duplicate sends.

FR-04

Batched DB writes

Outbound send results and webhook status updates are applied via a single `UPDATE … FROM (VALUES …)` per batch of up to 200 rows, executed by a dedicated write-batcher worker. No per-message `$transaction` on the hot path.

FR-05

Webhook ingestion in <10 ms

The web droplet's webhook handler writes the incoming Telnyx payload to a Redis stream and returns 200 immediately. The DB write is async via the write-batcher. Webhook bursts cannot starve the sender's connection pool.

FR-06

Inbound STOP handling

Inbound webhooks resolve the user via the `PhoneNumber` table (revolver-aware) with a legacy fallback to `User.telnyxPhoneNumber`. STOP messages set `Contact.subscribed = false` and `unsubscribeReason = 'STOP'`.

FR-07

Conversation table

A new `Conversation` model maintains per-contact summary state (last message text/time/direction, unread count, totals) updated on every message write. The messages page reads from this table directly — no JS-side grouping.

FR-08

Daily-stats rollup table

A new `MessageDailyStats` rollup table (keyed by user, campaign, phone, day) is refreshed every 5–15 min by a rollup worker reading from the replica. Dashboard and analytics queries scan O(days) rows, not O(messages).

FR-09

Redis-cached dashboard reads

Dashboard, message-stats, campaign-list, shortlink-list, and analytics-breakdown reads are wrapped with Redis caching (TTLs in §4). Cache busts on the matching mutations (send completion, campaign edit, etc.).

FR-10

Click codes generated at submission

Per-contact click codes are generated and inserted in bulk at campaign-fanout time, not at send time. Send workers never touch the `messageClickCode` table.

FR-11

Workflow / manual-fire on queue

The workflow executor's per-message send path also enqueues to the per-number Redis queues. The legacy `sendMessage()` in `src/lib/telnyx.js` is removed from the hot path.

FR-12

Job lifecycle

`SendJob` rows track campaign-level progress: `queued → fanout → sending → completed | failed`. Live counts are written to Redis during the send and flushed to Postgres every 5 s. Stuck-in-processing jobs >15 min auto-recover via BullMQ + cron.

FR-13

Dead-letter queue

Send jobs that exhaust 3 Telnyx retries land on `send:phone:{id}:dead`. UI shows DLQ depth per number and provides a one-click requeue or auto-unsubscribe action.

FR-14

Auto-unsubscribe on terminal failure

Permanent failure error codes (carrier-rejected, invalid number) trigger `Contact.subscribed = false` with `unsubscribeReason` set appropriately, gated by `User.autoUnsubscribeOnFailure`. Idempotent on retry.

FR-15

Settings UI parity

All existing settings (revolver phone numbers, rate limits, opt-out text, Telnyx config) remain editable from `/hermes/settings`. Adding a new phone number to the pool is a single form action.

FR-16

Schema migration safety

All schema changes (Conversation, MessageDailyStats, partitioning of `messages`, new indexes) are non-destructive and reversible. Existing data is backfilled via a one-shot script that can run in the background.

4 Non-Functional Requirements

Performance, reliability, observability, and operational targets the rebuild is held to.

CATEGORY	REQUIREMENT	TARGET / DEFINITION
Throughput	Aggregate sustained MPS	≥60 MPS across 3 toll-free numbers, sustained for the full duration of a 10,000-contact blast, measured by row count in `messages` grouped by `date_trunc('second', "sentAt")`.

CATEGORY	REQUIREMENT	TARGET / DEFINITION
Throughput	Per-number MPS ceiling	20 MPS per number, enforced at the Redis queue layer. No carrier throttling errors (Telnyx 40010/40011) during sustained sends.
Latency	Webhook receive → 200	p95 < 10 ms (received → response sent). The DB write is async and not part of this budget.
Latency	Dashboard first paint	p95 < 300 ms on the dashboard, messages, campaigns, and analytics pages with a tenant of 10M+ historical messages.
Latency	Send-worker job duration	p99 < 500 ms per message (queue pickup → DB write enqueued).
Reliability	Idempotency under retry	Zero duplicate outbound messages per `(campaignId, contactId)` across any number of process crashes / retries / cron recoveries.
Reliability	Job recovery	Any `SendJob` stuck in `processing` >15 min is automatically resumed; no job permanently lost on dyno restart.
Reliability	Data durability	Every outbound message has a row in `messages` with final status within 5 minutes of Telnyx's terminal webhook.
Cache TTLs	Dashboard summary	60 s, busted on send-job completion.
Cache TTLs	Message stats / counts	5 min, busted on send-job completion.
Cache TTLs	Campaign / shortlink lists	5 min, busted on create/update/delete of the underlying entity.
Cache TTLs	Analytics breakdown	10 min, scheduled refresh every 15 min by rollup worker.
Observability	Metrics	OpenTelemetry → Grafana (Cloud or self-host). Mandatory dashboards: MPS per phone number (1-min avg), BullMQ queue depth per queue, send-worker job duration p99, write-batcher lag (oldest unwritten event), Telnyx error rate by code, DB pool utilization, dashboard p95.
Observability	Alerts	Pages/email on: MPS <15 sustained for 5 min during active send, queue depth >5000 for 5 min, write-

CATEGORY	REQUIREMENT	TARGET / DEFINITION
		batcher lag >30 s, Telnyx 5xx rate >1%, DB pool >90%.
Operations	Deploy	One Docker image, deployed via DO App Spec or equivalent. New worker droplets come up with a single command and pick up work immediately.
Operations	Rollback	Any deploy can be rolled back in <5 min via image tag. No multi-step DB migrations in the deploy path; migrations run separately and are forward-compatible with the previous version.

5 Tech Stack & Infrastructure

Components to use. Where I'm flexible vs. firm.

Firm choices

- **Language:** Node.js 22 (Hermes today). Worker tier shares the same image as web.
- **Framework:** Next.js 14 App Router for the web tier (existing). Plain Node for worker processes.
- **ORM:** Prisma (existing). Hot-path bulk writes use raw `$executeRawUnsafe` with parameterized VALUES.
- **Database:** PostgreSQL 16+ (managed on DO). Primary + read replica. PgBouncer transaction-mode pooling.
- **Queue + cache:** Redis 7+ (managed on DO or Upstash). BullMQ as the queue library.
- **SMS provider:** Telnyx (existing). SDK v2 or raw `fetch()` if SDK gets in the way.
- **Hosting:** DigitalOcean — droplets behind a DO Load Balancer, managed Postgres, managed Redis, S3-compatible object storage (existing).
- **Auth:** NextAuth (existing).
- **CI/CD:** GitHub Actions → Docker registry → DO droplets (or DO App Platform during transition).

Recommended but flexible

- **Observability:** OpenTelemetry instrumentation. Grafana Cloud free tier is fine, Datadog or self-hosted Prometheus/Grafana also acceptable.
- **Process manager:** Docker + systemd on each droplet, or PM2 inside the container — implementer's call as long as restarts are reliable.
- **Migrations:** Prisma migrate via the `directUrl` path (port 25060 direct, not pooler).
- **Partitioning helper:** `pg_partman` or a small Node cron — implementer's call.

Not allowed

- No new primary database technology (Mongo, DynamoDB, Cassandra). Postgres only.
- No serverless functions (Lambda, Vercel functions) for the send path — long-running workers must be droplets or equivalent.
- No new SMS provider; Telnyx stays.
- No proprietary queue (SQS, Pub/Sub) — Redis + BullMQ is the standard.

6 Data Model Changes

What changes in the schema. The existing tables stay; this is purely additive plus targeted indexes.

New tables

- **`Conversation`** — per (user, contactPhone) summary with `lastMessageAt`, `lastMessageText` (truncated preview), `lastMessageDir`, `unreadCount`, `totalMessages`, `totalDelivered`, `totalFailed`. Unique on `(userId, contactPhone)`. Index on `(userId, lastMessageAt DESC)`.
- **`MessageDailyStats`** — rollup by (userId, campaignId, phoneNumberId, day) with sent / delivered / failed / inboundReplies / cost / revenue / clicks / opens.

Modified tables

- **`Message`** — partitioned by month on `createdAt` using native PG declarative partitioning. `telnyxMetadata` written only on failure, not success. Add a unique partial index on `(campaignId, contactId)` for outbound rows to enforce dedup at the DB level.
- **`Contact`** — add `@@unique([userId, phone])`, `@@index([userId, source])`, `@@index([userId, assignedPhoneNumberId])`. Add `pg_trgm` GIN indexes on `firstName`, `lastName`, `email`, `phone` for ILIKE search.
- **`Shortlink`** — add `@@index([userId, createdAt])`.
- **`PhoneNumber`** — no schema changes; `currentMonthMessages` now actually updated by the rollup worker.

Backfill scripts (one-shot)

- Populate `Conversation` from existing `messages` rows (keyset-paginated, rate-limited so it doesn't hammer the primary).
- Populate `MessageDailyStats` from history.
- Migrate existing `messages` rows into partitions (or accept that partitioning starts going forward and historical rows live in a single default partition for now — implementer's call, document the trade-off).

7 Suggested Phasing

Not a hard contract — sequencing guidance so we ship value incrementally and de-risk the riskiest changes last. The implementer can rearrange if there's a better path.

PHASE 0

Quick wins

~1 week

Bump pool to 80, raw VALUES UPDATE in the existing send path, drop double-write of `telnyxMetadata`, idempotency key on Telnyx calls, fix immediate-job recovery. ~15–30 MPS achievable on current infra. See `PHASE_0_PLAN.md` in repo.

PHASE 1

Reads + caches

~2 weeks

Stand up Redis + Postgres read replica. Build `Conversation` table + `MessageDailyStats` rollup worker. Wrap dashboard / analytics / list reads with Redis caching. Dashboard goes from seconds to <300 ms.

PHASE 2

Queue + workers

~2–3 weeks

BullMQ queues + send-worker + write-batcher + webhook-processor. Feature-flagged side-by-side with the legacy send path. Hit 60 MPS sustained on a flagged campaign.

PHASE 3+4

Migration + polish

~2–3 weeks

Unified Docker image with `ROLE` env, droplet provisioning, DNS cutover to droplets behind DO LB. Partition `messages`. Workflow executor migrates onto the queue. Decommission App Platform. Observability dashboards live.

Total: ~7–9 weeks for one developer working full-time. Solo, part-time, expect 12–16 calendar weeks.

8 Deliverables

Concrete artifacts I expect when the work is complete.

1. **Source code** — pull requests against the existing `hermes` repo, organized by phase. Each PR has a clear description, screenshots/before-after for any UI change, and a checklist of acceptance criteria from this SOW.
2. **Docker image** — single image, multi-role (`ROLE=web|send-worker|writer|rollup|fanout`). Published to a registry I control (DO Container Registry or GHCR).
3. **Infrastructure-as-code** — DO droplet provisioning via Terraform, Pulumi, or DO App Spec / `doctl` scripts. I should be able to bring up the full stack in a fresh DO account with one command.
4. **Schema migrations** — Prisma migration files for every schema change, plus the one-shot backfill scripts as separate runnable Node scripts in `scripts/`.
5. **Observability config** — Grafana dashboards as JSON, OpenTelemetry instrumentation configured, alerting rules defined. I should be able to import the dashboards into a fresh Grafana instance.
6. **Operational runbook** — a `RUNBOOK.md` in the repo covering: how to deploy, how to roll back, how to add a new send-worker droplet, how to inspect a stuck job, how to drain the DLQ, what to do if Redis is down, what to do if the primary DB fails over, how to scale a number's `rateLimit`.
7. **Load test script** — a reproducible script that fires a 10,000-contact campaign across 3 numbers and reports actual MPS, error rate, and tail latency. I can re-run it after every deploy.
8. **Architecture doc** — a short `ARCHITECTURE.md` capturing the final shape of the system (post-rebuild) so anyone joining later can orient quickly. References this SOW for the "why."

9 Acceptance Criteria

The end-state is accepted when every box is checked. No subjective judgment calls.

- **Throughput.** A 10,000-contact send across 3 toll-free numbers completes with ≥58 MPS sustained average and ≥60 MPS peak (1-second buckets), with zero `P2024` errors and <0.5% Telnyx error rate.
- **Per-number rate limiting.** Each number's outbound rate stays at ≤21 MPS in any 1-second window (small overshoot is fine; persistent overshoot is not). No Telnyx 40010/40011 carrier throttles during sustained sends.
- **Idempotency.** A test forces a worker crash mid-send. On recovery, no contact appears in `messages` twice for the same campaign (verified by SQL: zero rows in `SELECT (campaignId, contactId), count(*) FROM messages WHERE direction='outbound' GROUP BY 1 HAVING count(*) > 1`).
- **Dashboard.** Dashboard, messages, campaigns, and analytics pages load in <300 ms p95 on a tenant with 10M+ messages historical (load-tested with realistic seed data).
- **Webhook latency.** Telnyx webhook handler returns 200 in <10 ms p95 measured at the receiver, with no dropped events during a 60 MPS send.
- **Recovery.** A `SendJob` killed mid-flight (process SIGKILL) resumes within 15 min via cron + BullMQ retry and completes successfully.
- **Conversation parity.** Every conversation visible in the legacy JS-grouped view appears identically in the new `Conversation`-table-driven view. No conversations lost.
- **Migration safety.** Production data is migrated with zero message loss, zero contact loss, zero revenue-event loss. Verified by row-count and checksum diffs before/after.
- **Observability.** All 7 mandatory Grafana dashboards exist and show real data. All 5 alert rules are wired.
- **Rollback drill.** A simulated bad deploy can be rolled back in <5 min without data loss.
- **Runbook.** The runbook contains operational procedures for all 10 scenarios listed in Deliverable #6 and I can execute each procedure successfully on my own.

10 Out of Scope

What this project explicitly does *not* include. If any of these come up mid-project, treat as a change request.

- **New user-facing features.** No new campaign types, workflow node types, attribution models, or UI components beyond what's necessary to surface the Conversation table and the DLQ.
- **Multi-tenant rework.** The data model already supports multi-tenancy via `userId`; we're not adding tenant isolation, billing, or admin features.
- **Third-party integrations.** No Slack notifications, Zapier triggers, webhook outputs to external systems, or analytics platform integrations.
- **Mobile app.** Web only.

- **Compliance work.** No SOC 2, no GDPR audit, no HIPAA. Standard practices (TLS, secrets in env vars, no PII in logs) only.
- **Number provisioning automation.** Adding a new Telnyx number to the pool is still manual (Telnyx dashboard + Hermes settings UI).
- **A/B testing framework.** Out of scope.
- **Replacement of Telnyx or Postgres.** Not on the table.

11 Assumptions

- I (Carson) provide DO account access, Telnyx API credentials, the existing repo at `github.com/carsoncraig/hermes` (or equivalent), and a staging environment that mirrors prod.
- I am available for ~2 hours of synchronous review per week and async on Slack/email during business hours.
- The freelancer owns deployment of their own changes to staging. I own production cutover unless we agree otherwise.
- Existing tests in `vitest` stay green throughout. New tests are added for any new module with non-trivial logic (queue producers, write batcher, rollup worker, idempotency wrapper).
- Telnyx's rate limits, idempotency-key behavior, and webhook delivery semantics are as documented in their public API docs as of the project start date.
- Production has fewer than 10 concurrent campaign sends at any given time. (If this changes, queue sizing assumptions need to be revisited.)

12 What I Provide

- The existing Hermes codebase, fully documented (`HERMES_REBUILD_PROPOSAL.md` in repo has the detailed audit; `PHASE_0_PLAN.md` has Phase 0 spec).
- Read/write access to the DO account, Telnyx account, GitHub repo.
- A staging Postgres DB seeded with realistic test data (~100k contacts, ~1M historical messages) for load testing.
- Decisions on scope, priority, and trade-offs within 1 business day of being asked.
- Acceptance testing on each phase within 2 business days of PR readiness.

13 Open Questions

Things I want the implementer to weigh in on before kicking off, not blockers to start.

1. **Telnyx SDK vs. raw fetch.** The Node SDK v2 should support `Idempotency-Key` via its options object, but it's worth validating before committing. If the SDK is awkward, raw `fetch()` is preferred on the hot path — the workers don't need the SDK abstraction.

2. **BullMQ vs. alternative queue libs.** BullMQ is the default recommendation. If you have a strong preference for something else (`pg-boss`, NATS, etc.), make the case and I'll consider it. Hard no on SQS/Pub/Sub (vendor lock).
3. **Partitioning timing.** Partitioning `messages` can either backfill historical rows or start fresh going forward. I lean toward "start fresh, historical rows in a default partition we can split later" — but if you have a clean way to do it during the migration, I'm open.
4. **Webhook-receiver as a separate role?** Currently the diagram has webhooks landing on the web droplets. If load testing shows webhook bursts impacting UI latency, splitting webhook ingest into its own droplet role is an easy refactor — flag this if you see it.
5. **Number assignment heuristic.** Current code uses "least-loaded by contact count." Once `currentMonthMessages` is actually being maintained, we might want to use that instead (least-loaded by recent send volume). Implementer's call within Phase 1.
